

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 1**



Struct Dan Pointer

Oleh:

Vivaldi Fachmy Fahlevi

NIM. 2510817210017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL ?

Laporan Praktikum Algoritma & Struktur Data Modul 1: Struct Dan Pointer ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program	9
C. Pembahasan	10
SOAL 2.....	12
A. Source Code.....	12
B. Output Program.....	16
C. Pembahasan	17
TAUTAN GIT	19

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Soal 1	9
Gambar 2 Screenshot Output Program Soal 2.....	16

DAFTAR TABEL

Tabel 1 Source Code File point.h	6
Tabel 2 Source Code File line.h	7
Tabel 3 Source Code File line.cpp.....	7
Tabel 4 Source Code File prob1.cpp	8
Tabel 5 Source Code File point.h	12
Tabel 6 Source Code File circle.h	13
Tabel 7 Source Code File prob2.cpp	14
Tabel 8 Source Code File circle.cpp.....	15

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_P, y_P)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

A. Source Code

Tabel 1 Source Code File point.h

1	<code>#ifndef POINT_H</code>
2	<code>#define POINT_H</code>
3	
4	<code>struct Point {</code>
5	<code> double x;</code>
6	<code> double y;</code>
7	<code>};</code>
8	
9	<code>#endif</code>

Tabel 2 Source Code File line.h

1	#ifndef LINE_H
2	#define LINE_H
3	
4	#include "point.h"
5	#include <string>
6	
7	struct Line {
8	double a;
9	double b;
10	double c;
11	};
12	
13	inline constexpr double EPSILON = 1e-9;
14	
15	double gradient(const Line *l, const Point *p);
16	std::string CheckPointPosition(double value);
17	
18	#endif

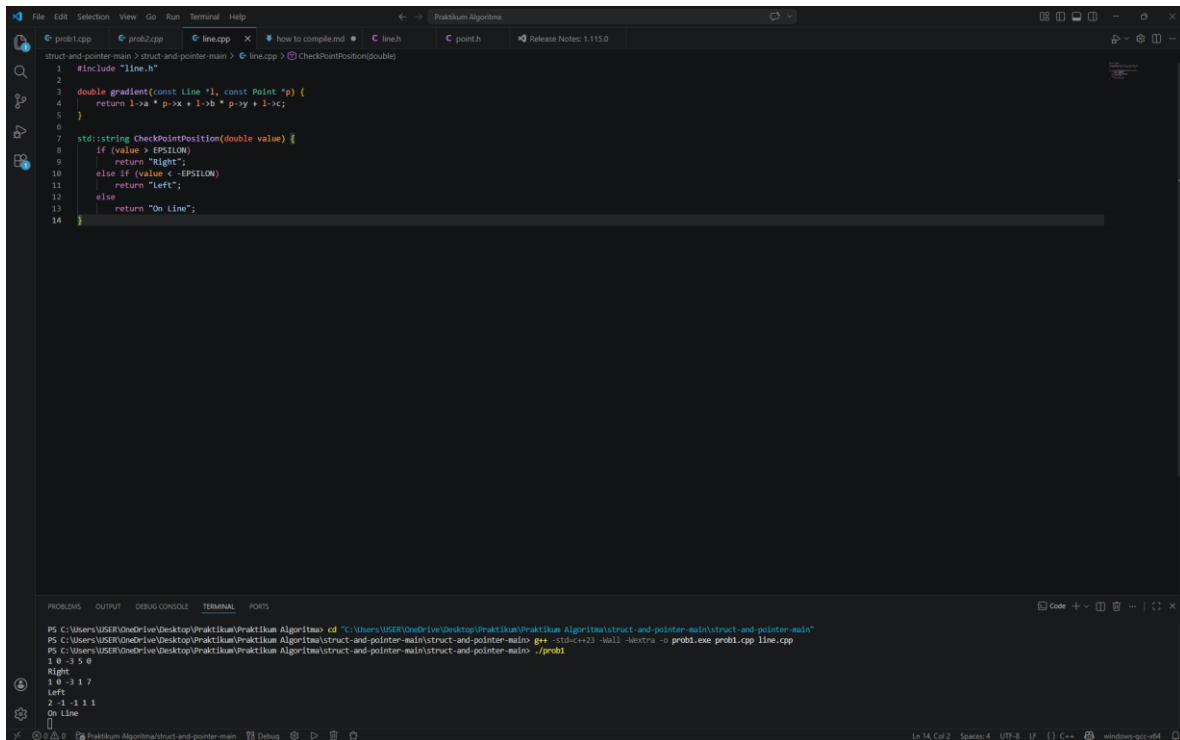
Tabel 3 Source Code File line.cpp

1	#include "line.h"
2	
3	double gradient(const Line *l, const Point *p) {
4	return l->a * p->x + l->b * p->y + l->c;
5	}
6	
7	std::string CheckPointPosition(double value) {
8	if (value > EPSILON)
9	return "Right";
10	else if (value < -EPSILON)
11	return "Left";
12	else
13	return "On Line";
14	}

Tabel 4 Source Code File probl.cpp

```
1  #include <iostream>
2  #include "line.h"
3  #include "point.h"
4
5  using namespace std;
6
7  int main() {
8      double a, b, c, px, py;
9      while (cin >> a >> b >> c >> px >> py) {
10         Line l = {a, b, c};
11         Point p = {px, py};
12
13         double hasil = gradient(&l, &p);
14         cout << CheckPointPosition(hasil) << endl;
15     }
16
17     return 0;
18 }
```


B. Output Program



The screenshot shows a C++ IDE with a dark theme. The editor displays the following code:

```
1 #include "line.h"
2
3 double gradient(const Line *l, const Point *p) {
4     return l->a * p->x + l->b * p->y + l->c;
5 }
6
7 std::string CheckPointPosition(double value) {
8     if (value > EPSILON)
9         return "Right";
10    else if (value < -EPSILON)
11        return "Left";
12    else
13        return "On Line";
14 }
```

The terminal window at the bottom shows the following output:

```
PS C:\Users\USER\OneDrive\Desktop\Praktikum\Praktikum Algoritma> cd "C:\Users\USER\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main\struct-and-pointer-main"
PS C:\Users\USER\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main> g++ -std=c++23 -Wall -std=c++11 struct-and-pointer-main\prob1.cpp prob1.cpp line.cpp
1 0 -3 5 0
Right
1 0 -3 1 7
Left
2 -1 -1 1 1
On Line
[]
```

Gambar 1 Screenshot Output Program Soal 1

C. Pembahasan

File point.h digunakan untuk mendefinisikan struktur data yang merepresentasikan sebuah titik dalam koordinat dua dimensi. Pada baris [1–2] terdapat *header guard* (`#ifndef`, `#define`) yang berfungsi untuk mencegah file ini diproses lebih dari satu kali saat proses kompilasi. Hal ini penting agar tidak terjadi duplikasi definisi. Pada baris [4–7] didefinisikan sebuah struct `Point` yang memiliki dua atribut yaitu `x` dan `y` bertipe `double`, yang merepresentasikan posisi titik pada sumbu koordinat. Terakhir, pada baris [9] terdapat `#endif` yang menutup *header guard*. File ini tidak memiliki fungsi, hanya sebagai penyedia tipe data yang akan digunakan oleh file lain.

File line.h berfungsi sebagai header yang mendeklarasikan struktur garis dan fungsi-fungsi yang akan digunakan untuk menentukan posisi titik terhadap garis. Pada baris [1–2] digunakan *header guard* untuk menghindari duplikasi. Baris [4] menyertakan file `point.h` karena struktur `Line` akan digunakan bersama dengan `Point`, dan baris [5] menyertakan pustaka `<string>` untuk penggunaan tipe data `string`. Pada baris [7–11] didefinisikan struct `Line` dengan tiga parameter yaitu `a`, `b`, dan `c` yang merepresentasikan persamaan garis umum $ax + by + c = 0$. Pada baris [13] dideklarasikan konstanta `EPSILON` yang digunakan sebagai toleransi untuk perbandingan bilangan desimal agar lebih akurat. Selanjutnya pada baris [15–16] terdapat deklarasi dua fungsi, yaitu `gradient()` untuk menghitung nilai substitusi titik ke persamaan garis, dan `CheckPointPosition()` untuk menentukan posisi titik berdasarkan hasil perhitungan tersebut.

File line.cpp berisi implementasi dari fungsi-fungsi yang telah dideklarasikan di `line.h`. Pada baris [1] dilakukan `#include "line.h"` agar file ini mengetahui deklarasi yang akan diimplementasikan. Fungsi `gradient` pada baris [3–5] menerima parameter pointer ke `Line` dan `Point`, lalu menghitung nilai dari persamaan $ax + by + c$ dengan cara mengakses nilai melalui pointer (`l->a`, `p->x`, dll). Nilai ini digunakan untuk menentukan posisi titik terhadap garis. Selanjutnya, fungsi `CheckPointPosition` pada baris [7–13] digunakan untuk mengklasifikasikan posisi titik. Jika nilai lebih besar dari `EPSILON`, maka titik berada di sebelah kanan garis (*Right*). Jika nilai lebih kecil dari negatif `EPSILON`, maka titik berada di sebelah kiri (*Left*). Jika nilainya mendekati nol (dalam

rentang EPSILON), maka titik berada tepat pada garis (*On Line*). Penggunaan EPSILON penting untuk menghindari kesalahan akibat pembulatan angka desimal.

File prob1.cpp merupakan program utama yang menjalankan keseluruhan logika. Pada baris [1–3] dilakukan *include* terhadap pustaka standar iostream serta file line.h dan point.h agar dapat menggunakan fungsi dan struktur yang telah dibuat sebelumnya. Baris [5] menggunakan using namespace std untuk mempermudah penulisan tanpa harus menambahkan std::. Pada baris [7] dimulai fungsi main() sebagai titik awal eksekusi program. Baris 8 mendeklarasikan variabel input yaitu a, b, c untuk garis, serta px, py untuk titik. Pada baris [9] digunakan perulangan while(cin >> ...) untuk membaca input secara berulang. Baris [10–11] membuat objek Line dan Point berdasarkan input yang diberikan. Pada baris [13] dilakukan pemanggilan fungsi gradient() untuk menghitung hasil substitusi titik ke persamaan garis. Kemudian pada baris [14] hasil tersebut dikirim ke fungsi CheckPointPosition() untuk menentukan posisi titik, dan hasilnya ditampilkan ke layar. Program akan terus berjalan selama masih ada input, dan diakhiri dengan return 0 pada baris [17].

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi distance dan CheckPointInCircle pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

A. Source Code

Tabel 5 Source Code File point.h

1	#ifndef LINE_H
2	#define LINE_H
3	
4	#include "point.h"
5	#include <string>
6	
7	struct Line {
8	double a;
9	double b;

Tabel 6 Source Code File circle.h

1	#ifndef CIRCLE_H
2	#define CIRCLE_H
3	
4	#include "point.h"
5	#include <string>
6	
7	struct Circle {
8	Point centre;
9	double radius;
10	};
11	
12	inline constexpr double EPSILON = 1e-9;
13	
14	double distance(const Circle *c, const Point *p);
15	std::string CheckPointInCircle(double distance,
16	double radius);
17	#endif

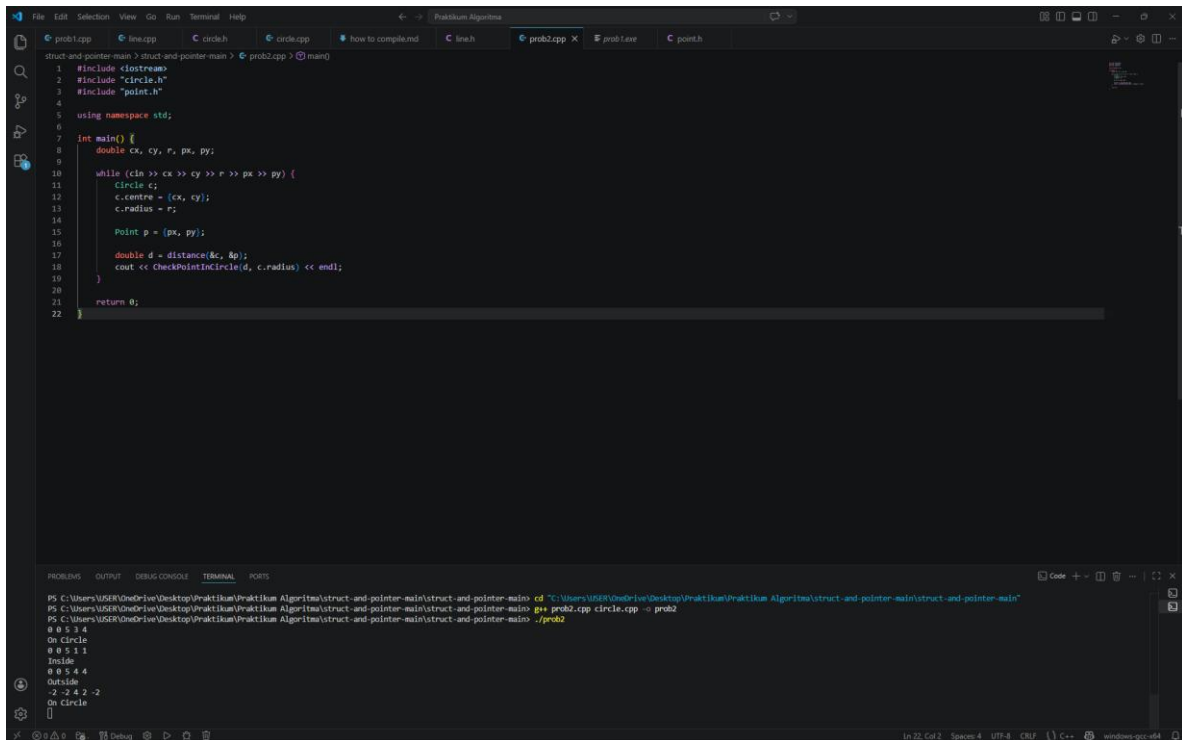
Tabel 7 Source Code File prob2.cpp

```
1  #include <iostream>
2  #include "circle.h"
3  #include "point.h"
4
5  using namespace std;
6
7  int main() {
8      double cx, cy, r, px, py;
9
10     while (cin >> cx >> cy >> r >> px >> py) {
11         Circle c;
12         c.centre = {cx, cy};
13         c.radius = r;
14
15         Point p = {px, py};
16
17         double d = distance(&c, &p);
18         cout << CheckPointInCircle(d, c.radius) <<
19         endl;
20     }
21
22     return 0;
23 }
```

Tabel 8 Source Code File circle.cpp

```
1  #include "circle.h"
2  #include <cmath>
3
4  double distance(const Circle *c, const Point *p) {
5      return sqrt((p->x - c->centre.x) * (p->x - c-
6      >centre.x) +
7      (p->y - c->centre.y) * (p->y - c-
8      >centre.y));
9  }
10
11 std::string CheckPointInCircle(double dist, double r) {
12     if (fabs(dist - r) <= EPSILON)
13         return "On Circle";
14     else if (dist < r)
15         return "Inside";
16     else
17         return "Outside";
18 }
```

B. Output Program



```
1 #include <iostream>
2 #include "circle.h"
3 #include "point.h"
4
5 using namespace std;
6
7 int main() {
8     double cx, cy, r, px, py;
9
10    while (cin >> cx >> cy >> r >> px >> py) {
11        Circle c;
12        c centre = {cx, cy};
13        c.radius = r;
14
15        Point p = {px, py};
16
17        double d = distance(&c, &p);
18        cout << CheckPointInCircle(d, c.radius) << endl;
19    }
20
21    return 0;
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main> cd "C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main"
PS C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main> g++ prob2.cpp circle.cpp -o prob2
PS C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main> ./prob2
0 0 5 3 4
On Circle
0 0 5 1 1
Inside
0 0 5 4 4
Outside
-2 -3 4 2 -2
On Circle
```

Gambar 2 Screenshot Output Program Soal 2

C. Pembahasan

File point.h digunakan untuk mendefinisikan struktur data yang merepresentasikan sebuah titik dalam koordinat dua dimensi. Pada baris [1–2] terdapat *header guard* (`#ifndef`, `#define`) yang berfungsi untuk mencegah file ini diproses lebih dari satu kali saat proses kompilasi. Hal ini penting agar tidak terjadi duplikasi definisi. Pada baris [4–7] didefinisikan sebuah struct `Point` yang memiliki dua atribut yaitu `x` dan `y` bertipe `double`, yang merepresentasikan posisi titik pada sumbu koordinat. Terakhir, pada baris [9] terdapat `#endif` yang menutup *header guard*. File ini tidak memiliki fungsi, hanya sebagai penyedia tipe data yang akan digunakan oleh file lain.

File circle.h merupakan header file yang digunakan untuk mendefinisikan struktur data lingkaran serta mendeklarasikan fungsi yang akan digunakan dalam program. Pada baris [1–2] terdapat *header guard* (`#ifndef` dan `#define`) yang berfungsi untuk mencegah file ini diproses lebih dari satu kali saat kompilasi. Pada baris [4] file `point.h` disertakan karena struktur `Circle` menggunakan tipe data `Point` sebagai pusat lingkaran, sedangkan pada baris [5] disertakan pustaka `<string>` untuk penggunaan tipe data string. Baris [7–10] mendefinisikan struct `Circle` yang terdiri dari `centre` (titik pusat lingkaran) dan `radius` (jari-jari lingkaran). Pada baris [12] terdapat konstanta `EPSILON` yang digunakan sebagai toleransi dalam perbandingan bilangan desimal agar hasil lebih akurat. Selanjutnya pada baris [14–15] terdapat deklarasi dua fungsi, yaitu `distance()` untuk menghitung jarak antara titik dengan pusat lingkaran, dan `CheckPointInCircle()` untuk menentukan posisi titik apakah berada di dalam, di luar, atau tepat pada lingkaran.

File circle.cpp berisi implementasi dari fungsi-fungsi yang telah dideklarasikan pada `circle.h`. Pada baris [1] dilakukan `#include "circle.h"` agar file ini dapat menggunakan struktur dan deklarasi fungsi yang telah dibuat sebelumnya, dan pada baris [2] ditambahkan `<cmath>` untuk menggunakan fungsi matematika seperti `sqrt` dan `fabs`. Fungsi `distance` pada baris [4–7] digunakan untuk menghitung jarak antara titik dan pusat lingkaran menggunakan rumus Pythagoras, yaitu :

$$d = \sqrt{(x_p - c_x)^2 + (y_p - c_y)^2}$$

Rumus ini berasal dari konsep jarak dua titik pada bidang kartesius, yaitu selisih koordinat x dan y dikuadratkan, kemudian dijumlahkan dan diakarkan. Implementasi pada kode menghitung bagian dalam akar terlebih dahulu, lalu menggunakan fungsi `sqrt()` untuk mendapatkan nilai akhirnya. Nilai jarak tersebut kemudian digunakan oleh fungsi `CheckPointInCircle` pada baris [9–15] untuk menentukan posisi titik terhadap lingkaran. Jika selisih antara jarak dan jari-jari sangat kecil (dalam batas EPSILON), maka titik berada tepat pada lingkaran (*On Circle*). Jika jarak lebih kecil dari jari-jari, maka titik berada di dalam lingkaran (*Inside*), dan jika lebih besar maka berada di luar lingkaran (*Outside*). Penggunaan EPSILON penting untuk menghindari kesalahan akibat pembulatan angka desimal.

File prob2.cpp merupakan program utama yang menjalankan seluruh logika pengecekan posisi titik terhadap lingkaran. Pada baris [1–3] dilakukan *include* terhadap pustaka `iostream` serta file `circle.h` dan `point.h` agar dapat menggunakan struktur dan fungsi yang telah dibuat. Pada baris [5] digunakan `using namespace std` untuk mempermudah penulisan kode. Baris [7] merupakan awal dari fungsi `main()` sebagai titik masuk program. Pada baris 8 dideklarasikan variabel input yaitu `cx`, `cy` sebagai pusat lingkaran, `r` sebagai jari-jari, serta `px`, `py` sebagai koordinat titik. Baris [10] menggunakan perulangan `while(cin >> ...)` untuk membaca input secara berulang. Pada baris [11–13] dibuat objek `Circle` dan diisi dengan nilai pusat dan radius, sedangkan pada baris [15] dibuat objek `Point` untuk titik yang akan dicek. Selanjutnya pada baris [17] dipanggil fungsi `distance()` untuk menghitung jarak titik ke pusat lingkaran, dan pada baris [18] hasil tersebut dikirim ke fungsi `CheckPointInCircle()` untuk menentukan posisi titik. Hasilnya kemudian ditampilkan ke layar. Program akan terus berjalan selama masih ada input, dan diakhiri dengan `return 0` pada baris [21].

TAUTAN GIT

<https://github.com/Fiivaldy/PRAKTIKUM-ALGORITMA-DAN-STRUKTUR-DATA.git>